Caso 6 — Gestión de incidentes en los laboratorios de una universidad

Adapter **Decorator** **Facade** **Factory Method** **Builder**

Duración: 2 horas **Lenguaje:** Java (SE 17+) **Modalidad:** individual, aplicación de consola

1. Contexto

La universidad cuenta con varios laboratorios de informática. Durante la jornada se presentan incidentes como equipos que se apagan, aplicaciones que dejan de responder y servicios de impresión que quedan fuera de línea. El personal de soporte recibe alertas desde herramientas diferentes: un monitor antiguo que envía mensajes de texto con campos separados por el carácter **|** y una herramienta reciente que entrega los datos estructurados.

Actualmente, cada técnico interpreta las alertas de una manera distinta. En ocasiones se registra dos veces el mismo incidente, se pierde la ubicación del equipo o se envía una notificación sin indicar su prioridad. Además, el coordinador debe ejecutar manualmente varios pasos para registrar, asignar y comunicar cada atención.

La coordinación de soporte solicita una aplicación que normalice las alertas, cree incidentes según su tipo, calcule su prioridad y genere órdenes de atención consistentes. El proceso debe poder ejecutarse desde un único punto de entrada, sin que la aplicación principal conozca los detalles internos de cada integración o notificación.

2. Reglas de negocio

2.1 Tipos de incidente

Tipo	Condición de clasificación	Área responsable	Tiempo objetivo
HARDWARE	Falla física, temperatura elevada o apagado inesperado del equipo.	Soporte de equipos	4 horas
SOFTWARE	Aplicación bloqueada, error de ejecución o sistema operativo inestable.	Soporte de software	8 horas
SERVICIO	Impresión, autenticación o servicio compartido no disponible.	Soporte de servicios	12 horas

2.2 Formato de alertas recibidas

Origen	Ejemplo de entrada	Interpretación
Monitor antiguo	`HOST|CPU|92|LAB-03|5`	Equipo LAB-03, evento CPU, valor 92, severidad 5.
Monitor moderno	`AlertData("LAB-02", "APP", "Aplicación congelada", 4, 3)`	Ubicación, categoría, descripción, severidad y usuarios afectados.

El sistema debe convertir ambas entradas a una estructura común AlertData con: ubicacion, categoria, descripcion, severidad (1 a 5) y usuariosAfectados (entero mayor o igual a 0). Para el monitor antiguo, el campo de evento CPU se clasifica como HARDWARE; APP como SOFTWARE; PRINT o AUTH como SERVICIO.

2.3 Cálculo de prioridad y SLA

Calcule un puntaje de atención con la siguiente fórmula:

puntaje = (severidad × 2) + mínimo(usuariosAfectados, 10)

Puntaje	Prioridad	Tiempo máximo de primera respuesta
17 a 20	CRÍTICA	1 hora
12 a 16	ALTA	2 horas
7 a 11	MEDIA	4 horas
0 a 6	BAJA	8 horas

El puntaje se calcula después de normalizar la alerta. La prioridad y el SLA deben quedar registrados en el incidente y en la orden de atención.

2.4 Validaciones

• La severidad debe estar entre 1 y 5; usuarios afectados no puede ser negativo.
• La ubicación, categoría y descripción son obligatorias.
• La categoría debe corresponder a HARDWARE, SOFTWARE o SERVICIO después de su normalización.
• La orden de atención debe tener identificador, fecha, incidente, técnico asignado y ubicación.
• La duración estimada debe ser mayor que cero cuando se indique.
• Si falta un campo obligatorio o se incumple una regla del Builder, build() debe lanzar IllegalStateException.

3. Requerimiento

Construya en Java una aplicación de consola que reciba alertas de dos formatos, las convierta a un modelo común, cree el incidente correspondiente, calcule su prioridad y genere una orden de atención. El proceso de registro debe poder invocarse mediante una única fachada.

4. Patrones obligatorios (exactamente cinco)

Adapter — integración del monitor antiguo

Defina una interfaz o contrato común para obtener una alerta normalizada. Implemente un adaptador que reciba el texto del monitor antiguo, separe sus campos y produzca un objeto AlertData. La lógica de negocio no debe interpretar directamente cadenas con el formato HOST|EVENTO|VALOR|UBICACION|SEVERIDAD. La entrada moderna puede convertirse directamente a AlertData.

Factory Method — creación de incidentes

Defina una clase creadora abstracta (por ejemplo, IncidentCreator) con el método createIncident(AlertData). Implemente creadores concretos para HARDWARE, SOFTWARE y SERVICIO. Cada creador debe instanciar el tipo concreto de incidente correspondiente. El código cliente debe trabajar con la abstracción Incident y no construir los incidentes mediante un switch o una cadena de if/else que contenga toda su creación.

Builder — orden de atención

Defina OrdenAtencion como una clase inmutable construida mediante un Builder fluido. Obligatorios: numeroOrden, fecha, incidente, tecnicoAsignado y ubicacion. Opcionales: descripcionTrabajo, duracionEstimadaMinutos, repuestos, observaciones y turno. build() valida los campos obligatorios y que la duración, si se suministra, sea positiva; ante incumplimiento lanza IllegalStateException.

Decorator — enriquecimiento de notificaciones

Defina una interfaz Notificacion con un método enviar(String mensaje). Implemente una notificación base que imprima el mensaje en consola. Añada al menos dos decoradores concretos que envuelvan otra Notificacion: uno que agregue la prioridad al mensaje (por ejemplo, [CRITICA]) y otro que agregue un registro de auditoría (por ejemplo, [INCIDENTE REGISTRADO]). Los decoradores deben poder combinarse sin modificar la clase base ni duplicar su código. No se requiere conexión real a correo, SMS ni servicios externos.

Facade — coordinación del registro

Implemente IncidentManagementFacade con un método público para registrar una alerta. La fachada debe coordinar la normalización (cuando la entrada sea del monitor antiguo), selección del creador concreto, creación del incidente, cálculo de prioridad, construcción de la orden y envío de la notificación decorada. La clase principal debe comunicarse con la fachada y no ejecutar por su cuenta cada paso del flujo.

5. Componente algorítmico obligatorio

1. Normalización: convertir la entrada antigua a AlertData y validar que contenga los campos requeridos.
2. Clasificación: determinar el tipo de incidente según la categoría del evento.
3. Puntaje y prioridad: aplicar la fórmula de la sección 2.3 y obtener la prioridad y su tiempo de respuesta.
4. Generación de orden: asignar un número consecutivo o identificador fijo, asociar técnico según el tipo de incidente y construir la orden.
5. Resumen: imprimir una tabla o listado con número de orden, ubicación, tipo, puntaje, prioridad, técnico y SLA.

6. Programa de demostración

1. Registre una alerta del monitor antiguo: **HOST|CPU|92|LAB-03|5**. Debe producir un incidente HARDWARE y una prioridad calculada.
2. Registre una alerta moderna de software para LAB-02, con severidad 4 y 3 usuarios afectados.
3. Registre una alerta antigua de servicio: **HOST|PRINT|0|LAB-01|3**.
4. Demuestre que las tres alertas generan órdenes con sus datos y técnicos correspondientes.
5. Envíe al menos una notificación combinando los dos decoradores y otra usando solo uno de ellos.
6. Intente construir una orden sin técnico asignado y otra con duración estimada igual a 0, dentro de try/catch, mostrando los mensajes de las excepciones.
7. Muestre el resumen final de órdenes registradas.

Formato sugerido de salida

```
=== SISTEMA DE SOPORTE - GESTION DE INCIDENTES ===
ALERTA NORMALIZADA | LAB-03 | HARDWARE | severidad 5 | usuarios 10
INCIDENTE: INC-001 | puntaje 20 | PRIORIDAD CRITICA | primera respuesta: 1 h
ORDEN: OT-001 | tecnico: Soporte de equipos | ubicacion: LAB-03
[CRITICA] Incidente INC-001 registrado en LAB-03
[INCIDENTE REGISTRADO] Auditoria: OT-001
[ERROR CONTROLADO] No se puede construir la orden: falta tecnico asignado
[ERROR CONTROLADO] La duracion estimada debe ser mayor que cero
RESUMEN: 3 incidentes registrados | 1 critica | 1 alta | 1 media
```

7. Restricciones y entregables

• Java estándar (SE 17+), sin frameworks ni librerías externas.
• Aplicación de consola; los datos de demostración pueden estar definidos directamente en el código.
• No se requiere persistencia, interfaz gráfica, conexión real con monitores, correo o SMS.
• Se deben implementar exactamente los cinco patrones indicados. No se permite añadir otros patrones como requisito obligatorio.
• Paquetes sugeridos: modelo, integracion, creacion, orden, notificacion, servicio y app.
• Entregue el proyecto comprimido (.zip), con código compilable y captura de la salida.
• Incluya PATRONES.md (máximo una página) con la ubicación de cada patrón, su propósito y una breve justificación de por qué corresponde a ese patrón.

8. Criterios de revisión

Criterio	Qué se verificará
Adapter	El formato antiguo se transforma a AlertData sin filtrar su estructura hacia la lógica de negocio.
Factory Method	Los creadores concretos encapsulan la instanciación de cada tipo de incidente.
Builder	La orden es inmutable y build() controla las reglas de construcción.
Decorator	Los decoradores envuelven una Notificacion y se pueden combinar.
Facade	La aplicación principal ejecuta el flujo mediante una única operación de alto nivel.
Reglas y demostración	Se calculan correctamente puntaje, prioridad, SLA y se muestran los errores controlados.

Fuera de alcance

Interfaz gráfica, base de datos, concurrencia, integración real con herramientas de monitoreo, envío real de correos o mensajes, autenticación, inventario de repuestos, asignación automática avanzada y pruebas automatizadas.

Nota de alcance: el ejercicio está dimensionado para una implementación individual de 2 horas. Se limita a dos formatos de entrada, tres tipos de incidente, dos decoradores y un flujo principal de registro. La prioridad se calcula con una fórmula directa para que el esfuerzo se concentre en la correcta aplicación de los patrones.